python_basics_for_numerical_analysis

October 5, 2026

1 Python Basics for Numerical Analysis

1.0.1 Beginner Exercise Notebook

This notebook introduces basic Python instructions that students will need later in a **Numerical Analysis course**.

Topics covered: - Variables and arithmetic operations - Input and output - Writing mathematical functions - Loops - Lists and simple numerical tables - A small numerical example

Students should **run the cells one by one

1.1 Exercise 1: Basic Arithmetic

Numerical Analysis relies heavily on mathematical operations.

Complete the missing operations.

[14]:
```
a = 10
b = 4

# TODO: complete the following operations
addition = a + b
subtraction = a - b
multiplication = a*b
division = a/b

print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)
```

```
Addition: 14
Subtraction: 6
Multiplication: 40
Division: 2.5
```

1.2 Exercise 2: Writing a Mathematical Function

In numerical methods we often evaluate mathematical functions.

Example function:

$f(x) = x^2 - 4x + 1$

Complete the function.

[15]:
```
def f(x):
    return x**2-4*x+1

x = 2
print("f(2) =", f(x))
```

```
f(2) = -3
```

1.3 Exercise 3: Using a Loop

Loops are used in numerical algorithms to perform **iterations**.

Complete the loop to compute squares of numbers.

[16]:
```
# Print squares of numbers from 1 to 5

for i in range(1,6):
    square = i*i
    print(i, "squared =", square)
```

```
1 squared = 1
2 squared = 4
3 squared = 9
4 squared = 16
5 squared = 25
```

1.4 Exercise 4: Creating a Numerical Table

In numerical analysis we often create tables of values of a function.

the following code to generate a table of f(x) for x from 0 to 5.

[17]:
```
print("x    f(x)")
print("---------")

for x in range(0,6):
    fx = f(x)
    print(x, "   ", fx)
```

```
x    f(x)
---------
0    1
1    -2
2    -3
3    -2
4    1
5    6
```

2 Using List

[18]:
```
# Step 1: Define the function
def f(x):
    return x**2 + 1

# Step 2: Create a list to store function values
function_values = []

# Step 3: Loop over x from 0 to 5
for x in range(0, 6):
    y = f(x)            # compute f(x)
    function_values.append(y)  # store it in the list

# Step 4: Print the list of function values
print("Function values:", function_values)
```

```
Function values: [1, 2, 5, 10, 17, 26]
```

2.1 Exercise 6: Simple Numerical Approximation

Compute the **average of three numbers** entered by the user.

This type of calculation appears frequently in numerical methods.

[19]:
```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

# TODO: compute the average
average = (a+b+c)/3

print("Average =", average)
```

```
Enter first number:  5
Enter second number:  6
Enter third number:  7

Average = 6.0
```

2.2 Evaluate the function

$f(x) = x^3 - x - 2$

for values of **x from 1 to 5** and print the results.

[20]:
```
# COMPLETE THE CODE

def f(x):
    return x**3-x-2
```

```
for x in range(1,6):
    print("x =", x, "  f(x) =", f(x))
```

```
x = 1   f(x) = -2
x = 2   f(x) = 4
x = 3   f(x) = 22
x = 4   f(x) = 58
x = 5   f(x) = 118
```

[21]:
```
# Import NumPy library
# NumPy is used for numerical computations and creating arrays of numbers
import numpy as np

# Import matplotlib library for plotting graphs
# pyplot provides functions similar to MATLAB plotting commands
import matplotlib.pyplot as plt


# ------------------------------------------------------
# Define the first mathematical function
# f(x) = x^2
# The function receives x as input and returns x squared
# ------------------------------------------------------
def f(x):
    return x**2


# ------------------------------------------------------
# Define the second mathematical function
# g(x) = 2x + 1
# This is a linear function
# ------------------------------------------------------
def g(x):
    return 2*x + 1


# ------------------------------------------------------
# Generate x values
# np.linspace(start, end, number_of_points)
# This creates 100 equally spaced values between -5 and 5
# More points make the graph smoother
# ------------------------------------------------------
x = np.linspace(-5, 5, 100)


# ------------------------------------------------------
```

```
# Evaluate the functions for all x values
# Because NumPy supports vector operations,
# the functions are applied to all elements of x
# ------------------------------------------------------
y1 = f(x)   # compute f(x) for each x
y2 = g(x)   # compute g(x) for each x


# ------------------------------------------------------
# Plot the first function
# x values on the horizontal axis
# y1 values on the vertical axis
# label is used later in the legend
# ------------------------------------------------------
plt.plot(x, y1, label="f(x) = x^2")


# ------------------------------------------------------
# Plot the second function on the same graph
# This allows us to visually compare the two functions
# ------------------------------------------------------
plt.plot(x, y2, label="g(x) = 2x + 1")


# ------------------------------------------------------
# Draw horizontal and vertical axes
# These help visualize where functions cross the axes
# ------------------------------------------------------
plt.axhline(0)   # x-axis
plt.axvline(0)   # y-axis

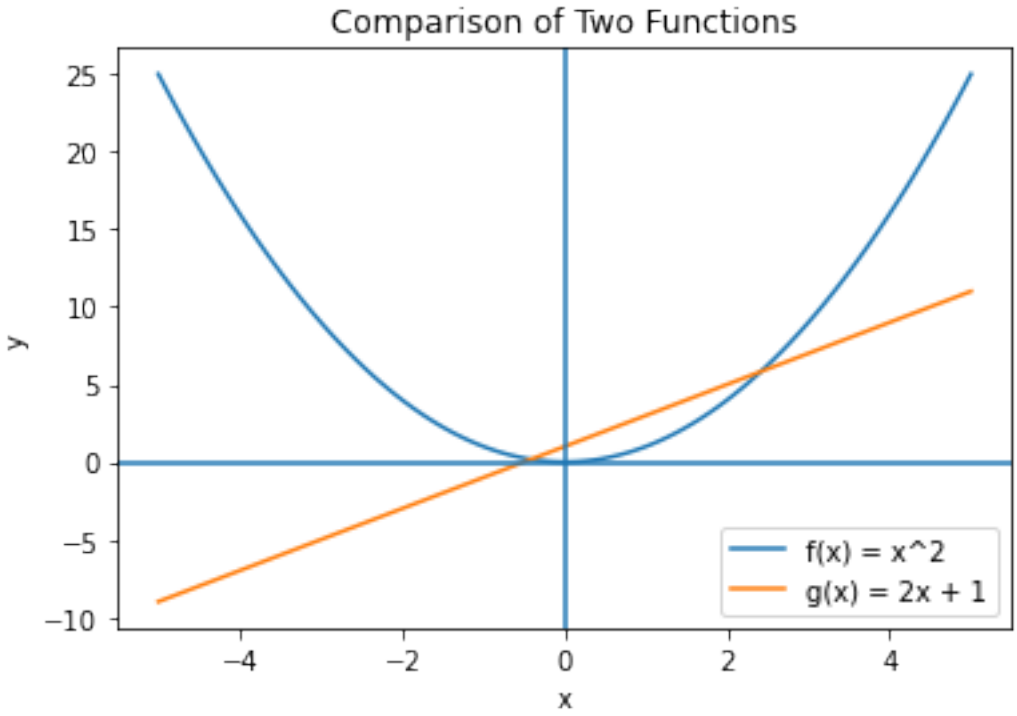

# ------------------------------------------------------
# Label the axes
# These describe what each axis represents
# ------------------------------------------------------
plt.xlabel("x")
plt.ylabel("y")


# ------------------------------------------------------
# Add a title to the graph
# ------------------------------------------------------
plt.title("Comparison of Two Functions")


# ------------------------------------------------------
```

```
# Display the legend
# The legend shows which curve corresponds to each function
# ------------------------------------------------------
plt.legend()


# ------------------------------------------------------
# Display the graph on the screen
# ------------------------------------------------------
plt.show()
```



3 Code Reusability

A general plotting function allows you to reuse the same code for many different functions.

Instead of writing plotting code every time:

[22]:
```
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt


# ------------------------------------------------------
```

```
# General plotting function
# This function receives:
# 1. func  -> the mathematical function to plot
# 2. x_min -> starting value of x
# 3. x_max -> ending value of x
# 4. n     -> number of points used for plotting
# ------------------------------------------------------
def plot_function(func, x_min, x_max, n=100):

    # Generate x values between x_min and x_max
    x = np.linspace(x_min, x_max, n)

    # Evaluate the function for all x values
    y = func(x)

    # Plot the function
    plt.plot(x, y)

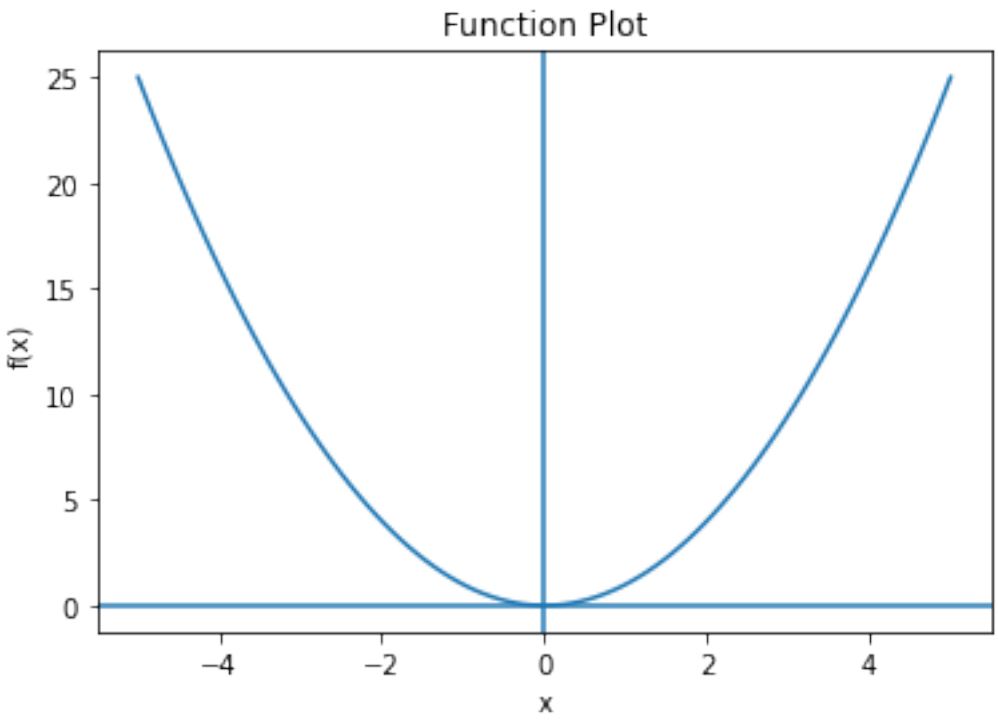
    # Draw axes
    plt.axhline(0)
    plt.axvline(0)

    # Labels and title
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.title("Function Plot")

    # Show the graph
    plt.show()
```

[23]:
```
# Example 1
def f(x):
    return x**2

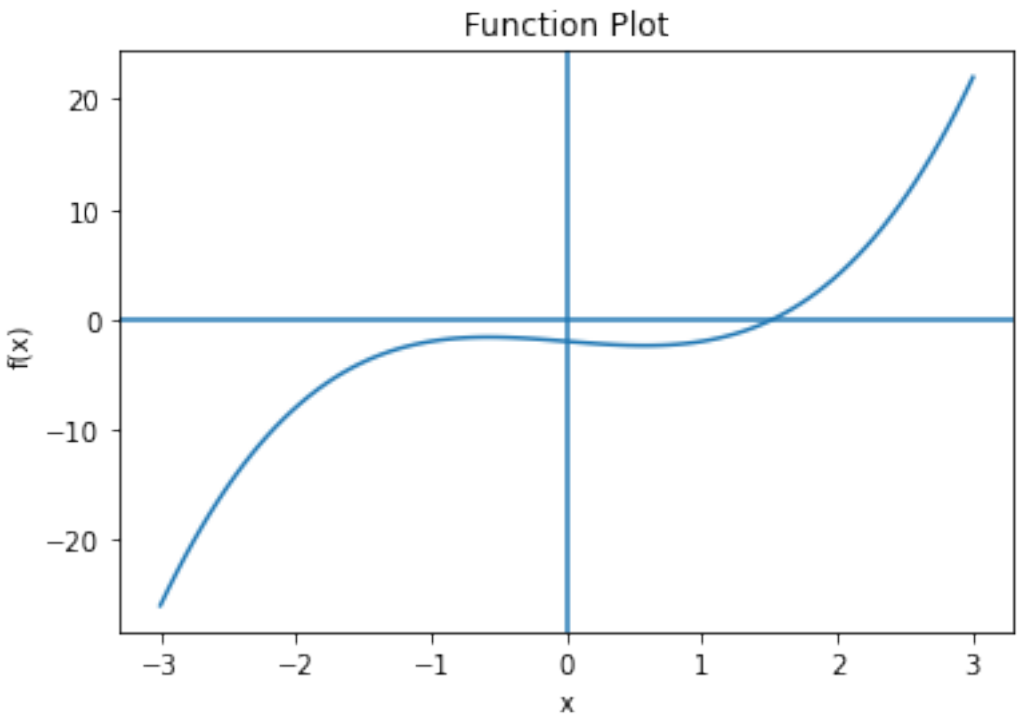
plot_function(f, -5, 5)
```



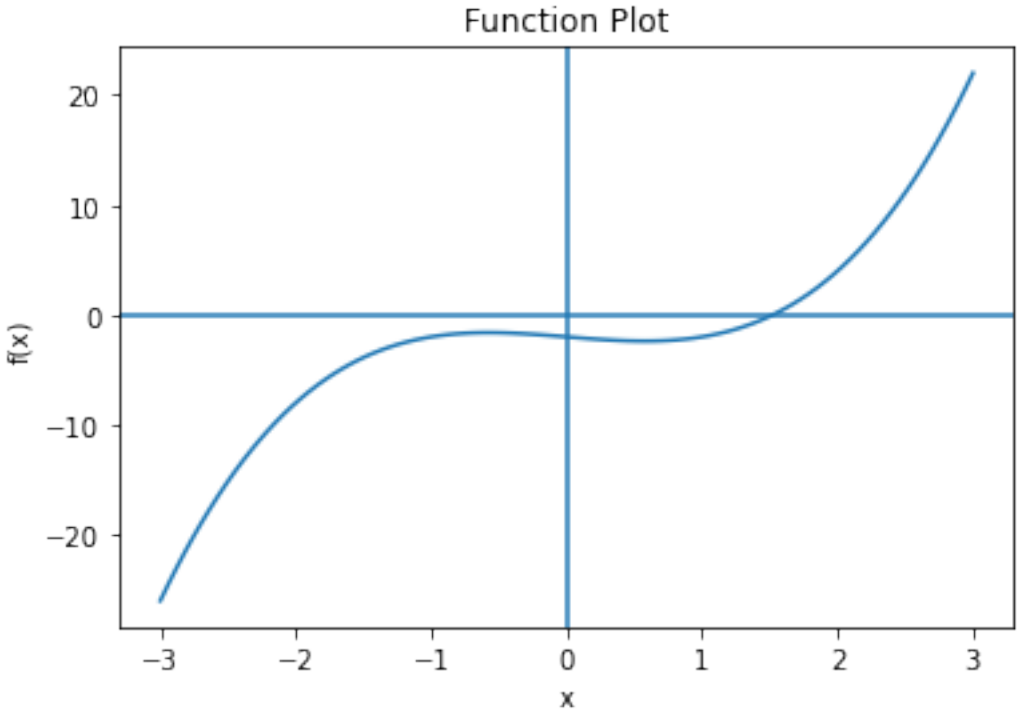
[24]:
```
# Example 2

def g(x):
    return x**3 - x - 2

plot_function(g, -3, 3)
```



[25]:
```
plot_function(lambda x: x**3 - x - 2, -3, 3) # another way of defining a␣
↪function
```



3.1 What You Learned

After completing this notebook, you should understand:

- Basic Python syntax
- Mathematical operations
- Writing functions
- Loops and iterations
- Creating tables of values
- Ploting a function

These skills are essential before learning numerical algorithms such as:

- Bisection Method
- False Position Method
- Newton Method
- Numerical Integration